Stitchwort: from source to a running program

commit a5cce012d · 2026-11-09

1 Source

`*.sw` · `stitchwort.toml`

A file or a project (namespaces, imports). The standard library is `.sw` too, shipped inside `elab` (`src/std/`).

↓

2 Parse

`crates/syntax`

- Lex and parse into the **surface AST**, with source spans.
- Syntax errors and their notes come from here; `stitchwort fmt` works on this tree.

↓ surface AST

3 Elaborate

`crates/elab`

Turns what you wrote into fully explicit **core terms**:

- names and imports resolved; dot calls, operators, sugar expanded;
- types inferred and checked; implicit arguments solved by unification (metavariables);
- `match` compiled to eliminators; records, ranges, clause lambdas desugared;
- abilities: `handle … with` becomes `handlePlus (A.%elim clauses) forward`;
- proofs: tactics (`rewrite`, `cases`, `omega`, …), function equations, termination measures.

Most error messages and goal printing live here.

↓ core terms

4 Kernel check

`crates/core`

The small **trusted** checker re-checks every definition and data group: types, termination (size-change graphs), positivity, universes. Accepted definitions get a **content hash**; the check cache is keyed by it.

- `stitchwort check`, `test`, `audit` and the language server stop here.
- `core` also has the **reference evaluator**: slow, but the check-time semantics.

Trust boundary. Everything after this point is **untrusted**: a bug can print a wrong answer but can never make a false proof check. Runtimes never depend on `elab`; differential tests compare them with the reference evaluator. Running a program continues at **5** (right), or at **7c** / **7d**.

Shared plumbing

- `crates/driver`: the pipeline itself (sessions, caching, tests, error rendering), shared by the CLI and the playground.
- `crates/cli`: the `stitchwort` command. `crates/lsp`: the language server (VS Code and the playground editor).

from 4, checked definitions ↓

5 Erase and lower

`crates/rt` `lower.rs`

Checked terms become the runtime's **IR**:

- types, proofs and implicit arguments erased;
- lambdas → closures, partial calls → partial applications, case trees → `match`;
- std functions swapped for **natives** (Nat arithmetic, Text, arrays), bound by content hash.

↓ IR

6 Optimise the IR

`crates/rt`

Simplifier (`opt.rs`), specialisation, state-handler fusion, tail recursion modulo cons, hoisting, and **in-place reuse** of cells nothing else holds (`reuse.rs`). Shared by the machine and native code.

↓

7a Bytecode + machine

`rt` `code.rs` · `machine.rs`

IR compiled to register bytecode, run by the machine on compact values with its own frame stack; effects run by the effect loop (`effect.rs`).

- `stitchwort run` (the default runtime);
- `stitchwort build`: a standalone executable (bytecode + the `standalone` runtime, linked with `cc`);
- the **playground**: the whole compiler and this machine compiled to **WebAssembly** (`crates/web`).

7b Native code

`crates/native (Cranelift)`

The same bytecode bodies compiled to machine code, linked with the runtime library (`value.rs`, `effect.rs`, `native_abi.rs`, mimalloc).

- `stitchwort run --native`, `stitchwort build --native` (what the speed comparisons measure);
- not in the WebAssembly build.

7c JavaScript

`crates/js`

Its own erasure and code generation straight from checked terms (no IR), plus TypeScript declarations and source maps.

- `stitchwort run --js` (runs with Node), `stitchwort build --js` (an ES module).

7d Reference evaluator

`crates/core`

`stitchwort run --reference`: evaluates core terms directly. Slow; the yardstick the others are tested against.

Which path a command takes

Command	Stages	Runs on
`stitchwort check` / `test` / `audit`	1 → 4 (tests then run on 5 → 7a)	—
`stitchwort run`	1 → 4 → 5 → 6 → 7a	machine
`stitchwort run --native` / `build --native`	1 → 4 → 5 → 6 → 7b	native code
`stitchwort build`	1 → 4 → 5 → 6 → 7a, linked into an executable	machine, standalone
`stitchwort run --js` / `build --js`	1 → 4 → 7c	Node / your JS
`stitchwort run --reference`	1 → 4 → 7d	reference evaluator
Playground (browser)	1 → 4 → 5 → 6 → 7a, all in WebAssembly	machine in WASM

Words on this page

Surface AST: the program as written, as a tree with spans.

Core term: the fully explicit form the kernel checks; what hashes and proofs are about.
Content hash: a definition's identity from its checked term; keys the check cache and binds natives.
Kernel: the trusted checker in `crates/core`; the only part a proof's validity depends on.
Reference evaluator: evaluates core terms directly; defines what a program means.

Erasure: dropping what has no run-time meaning (types, proofs, implicit arguments).
IR: the runtime's erased form: closures, partial applications, matches, natives.
Bytecode: the IR compiled to register instructions; the machine runs it, native code compiles it.
Effect loop: the runtime part that routes an ability's operations to their handler.
In-place reuse: overwriting a cell nothing else holds instead of freeing it and allocating.